MODUL PEMBELAJARAN

PHP & MySQL — Jembatan Aplikasi Dinamis

Pertemuan 11 | Edisi Revamp Khusus: Menyelaraskan Teori Modul & Implementasi Riil di Repositori Anda

🚀 Pengantar & Kenapa Modul ini Ditulis Ulang?

Selamat datang di modul Pertemuan 11 yang telah disempurnakan! Modul ini dirancang khusus untuk Anda yang sedang belajar di bootcamp namun merasa bingung karena **materi teoretis dari mentor/modul standar seringkali berbeda dengan apa yang Anda praktikkan di dunia nyata (di repositori GitHub Anda `supermarket-php`)** [1].

Mengapa Kode Anda di Repositori Berbeda?

Jika Anda membandingkan modul standar dengan kode Anda, ada perbedaan paradigma yang sangat mencolok:

1. **Paradigma Klasik (Procedural) vs. Modern (Object-Oriented/OOP)**: Modul lama mengajarkan fungsi seperti `mysqli_connect()`, sedangkan kode Anda menggunakan `new mysqli()` [8].
2. **Cara Pengambilan Data (*Fetching*)**: Modul mengajarkan perulangan `while` baris demi baris, sedangkan kode Anda langsung menarik semua data ke memori sebagai array associative utuh dengan `fetch_all()` lalu merendernya dengan `foreach` [12].
3. **Keamanan & Fitur Lanjutan**: Kode Anda di repositori ternyata sudah melompat jauh ke depan dengan menerapkan **Prepared Statements** (mencegah SQL Injection) [21] dan **Upload Gambar Fisik** [21] yang sebenarnya baru diajarkan di Pertemuan 13 & 14!

Modul ini hadir sebagai **jembatan** [2]. Kami akan membahas materi dasar Pertemuan 11 (Koneksi → Read → Create) [1] namun ditulis menggunakan **standar OOP modern yang Anda pakai di repositori** agar Anda memiliki panduan yang sinkron dan mudah dipahami.

🗺️ Peta Ruang Lingkup Materi (Pertemuan 11)

Hari ini kita membatasi pembahasan pada 3 pilar utama:

1. **Koneksi Database** (`config/koneksi.php`) — Membuka gerbang PHP ke MySQL [8].
2. **Menampilkan Data / Read** (`produk/index.php`) — Mengambil data dari database dan menampilkannya di halaman web [14].

3 **Menyimpan Data Baru / Create** (`produk/create.php` & `proses/store.php`) — Menerima input dari form browser dan menyimpannya secara aman ke database [16].

Bagian 1: Struktur Proyek yang Rapi

Untuk membangun aplikasi web tingkat industri, kita tidak bisa menumpuk semua kode dalam satu file. Kita memisahkan kode berdasarkan fungsinya agar mudah dikelola (*maintainability*) [7].

Berikut adalah struktur folder yang Anda terapkan di repositori **`supermarket-php`**, yang merupakan standar yang sangat bagus [7]:

```
supermarket-php/
├── assets/                 # Tempat menyimpan file statis (gambar, stylesheet)
│   └── produk/             # Folder khusus hasil upload gambar produk
├── config/
│   └── koneksi.php         # Gerbang utama koneksi database
├── templates/
│   ├── header.php          # Template bagian atas (Navbar, Meta HTML)
│   └── footer.php          # Template bagian bawah HTML
├── produk/                 # Modul khusus pengelolaan produk
│   ├── proses/
│   │   ├── store.php       # Script pemroses tambah data (tanpa HTML)
│   │   ├── update.php      # Script pemroses edit data
│   │   └── delete.php      # Script pemroses hapus data
│   ├── create.php          # Form tampilan tambah produk
│   ├── edit.php            # Form tampilan edit produk
│   └── index.php           # Halaman daftar produk (Tabel Admin)
├── index.php               # Halaman utama / Landing Page Etalase
└── style.css               # Kustomisasi tampilan CSS
```

> **Aturan Emas Pathing:**
> File di dalam folder seperti **`produk/index.php`** memanggil file konfigurasi atau template dengan menggunakan **`../`** (artinya naik satu tingkat folder) [7]. Contoh:
> **`require_once __DIR__ . '/../config/koneksi.php';`**

Bagian 2: File Koneksi (`config/koneksi.php`)

Koneksi adalah jembatan komunikasi. Di modul standar, koneksi ditulis secara procedural. Namun, di repositori Anda, koneksi ditulis menggunakan pendekatan **Object-Oriented Programming (OOP) menggunakan objek `mysqli`** [8].

Berikut adalah kode koneksi Anda yang sudah sangat tepat:

```
<?php
// config/koneksi.php
// Digunakan untuk konfigurasi database yang digunakan oleh aplikasi
```

```
$hostname = 'localhost'; // Server database lokal Anda
$username = 'root';      // Username default XAMPP
$password = '';          // Password default XAMPP (kosong)
$database = 'belajar_sql'; // Nama database yang Anda gunakan
$port     = 3306;        // Port MySQL default

// Membuat koneksi menggunakan Object-Oriented Style
$conn = new mysqli($hostname, $username, $password, $database, $port);

// Memeriksa apakah koneksi berhasil atau gagal
if ($conn->connect_error) {
    die("Connection to database failed: " . $conn->connect_error);
}
?>
```

Bedah Kode: Procedural vs OOP

Mari kita bandingkan secara adat cara kerja keduanya agar Anda paham mengapa cara Anda di repositori jauh lebih modern [8]:

Fitur	Pendekatan Procedural (Modul Lama)	Pendekatan OOP (Repositori Anda)
Pembuatan Koneksi	`conn = mysqli_connect(host, user, pass, db);`	`conn = new mysqli(host, user, pass, db, port);`
Cek Error	`if (!conn) { die(mysqli_connect_error()); }`	`if (conn->connect_error) { die(conn->connect_error); }`
Penggunaan Variabel	`conn` dikirim sebagai argumen di setiap fungsi [9].	`conn` bertindak sebagai objek mandiri yang memiliki metode sendiri.

> **Variabel `conn` itu apa?**
> `conn` bukanlah data produk. Ia adalah **saluran pipa aktif** ke MySQL server Anda [9]. Setiap kali PHP ingin meminta data, ia harus menggunakan saluran pipa `conn` ini [9].

Bagian 3: Read — Menampilkan Katalog Produk (`produk/index.php`)

Menampilkan data adalah proses "panen" [1]. Setelah data dimasukkan ke dalam database melalui phpMyAdmin, saatnya kita menampilkannya ke halaman web.

Konsep Penting: Cara Mengambil Data (*Fetching*)

Di sinilah letak perbedaan terbesar yang sering membuat siswa bingung.

A. Metode Modul Standar: `while` + `mysqli_fetch_assoc`

Modul lama mengajarkan pengambilan data sebaris demi sebaris di dalam perulangan `while` [12]:

```
while ($row = mysqli_fetch_assoc($result)) {
    // Tampilkan data baris demi baris
}
```

- **Kelebihan**: Sangat hemat memori karena data tidak dimuat sekaligus. Jika database berisi 1 juta baris, PHP hanya memproses 1 baris di memori pada satu waktu [12].
- **Kekurangan**: Sintaksnya terasa kurang intuitif bagi pemula karena struktur perulangan bercampur langsung di dalam render HTML [12].

B. Metode Repositori Anda: `fetch_all(MYSQLI_ASSOC)` + `foreach`

Di repositori Anda, data langsung diambil secara utuh ke dalam memori PHP sebagai array multi-dimensi [12]:

```
$result = $conn->query($sql)->fetch_all(MYSQLI_ASSOC);
```

Lalu di bagian tampilan, Anda cukup merendernya menggunakan perulangan `foreach` yang sudah sangat akrab bagi Anda sejak Pertemuan 9 [3, 5]:

```
foreach ($result as $p) {
    // Tampilkan produk
}
```

- **Kelebihan**: Pemisahan logika pengambilan data dan render tampilan menjadi sangat bersih. Anda bisa mengambil data di bagian paling atas file, lalu menutup tag PHP, dan baru menulis perulangan `foreach` murni di dalam HTML [15].
- **Kekurangan**: Kurang efisien jika data yang ditarik berukuran sangat besar (ratusan ribu baris) karena semuanya dipaksa masuk ke memori RAM server sekaligus [12]. Namun, untuk aplikasi toko berskala kecil-menengah, **cara ini adalah yang paling direkomendasikan karena kerapian kodenya.**

Implementasi Kode: `produk/index.php`

Berikut adalah kode halaman daftar produk Anda yang bersih dan terstruktur [14]:

```
<?php
// 1. SETUP & IMPORT KONEKSI
require __DIR__ . '/../config/koneksi.php';
require __DIR__ . '/../templates/header.php';

// 2. QUERY DATABASE
// Kita mengambil semua produk dari database belajar_sql
$sql = "SELECT * FROM products";
$result = $conn->query($sql)->fetch_all(MYSQLI_ASSOC);
?>

<!-- LINK STYLESHEET KUSTOM -->
```

```
<link rel="stylesheet" href="/../style.css">

<div class="container mt-4">
    <h1 style="text-align: center;" class="mb-4">Daftar Produk</h1>

    <!-- Bagian Tambah Produk -->
    <div class="mb-3 text-end">
        <a href="/produk/create.php" class="btn btn-success">
            <i class="bi bi-plus-circle"></i> Tambah Produk Baru
        </a>
    </div>

    <!-- Validasi Empty State: Menangani jika database kosong -->
    <?php if (count($result) === 0): ?>
        <div class="alert alert-warning text-center">
            Belum ada produk yang tersedia di katalog [15].
        </div>
    <?php else: ?>
        <table class="table table-striped table-bordered align-middle">
            <thead class="table-dark text-center">
                <tr>
                    <th style="width: 5%;">No</th>
                    <th>Nama Produk</th>
                    <th style="width: 20%;">Harga</th>
                    <th style="width: 10%;">Stock</th>
                    <th style="width: 15%;">Action</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($result as $p): ?>
                <tr>
                    <td class="text-center"><?= htmlspecialchars($p['id']); ?></td>
                    <td style="text-align: left;"><?= htmlspecialchars($p['nama']); ?></td>
                    <td>
                        <div style="display: flex; flex-direction:row; justify-content:space-between;">
                            <span>Rp </span>
                            <span><?= number_format($p['harga'], 0, ',', '.'); ?></span>
                        </div>
                    </td>
                    <td class="text-center"><?= htmlspecialchars($p['stock']); ?></td>
                    <td>
                        <div class="d-flex gap-2 justify-content-center">
                            <!-- Tombol Edit & Delete -->
                            <a href="/produk/edit.php?id=<?= $p['id']; ?>" class="btn btn-warning btn-sm text-white
                                <i class="bi bi-pencil-square"></i>
                            </a>
                            <a href="/produk/proses/delete.php?id=<?= $p['id']; ?>" class="btn btn-danger btn-sm" o
                                <i class="bi bi-trash-fill"></i>
                            </a>
                        </div>
                    </td>
                </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    <?php endif; ?>
</div>

<?php
```

```
// 3. FOOTER TEMPLATE
require __DIR__ . '/../templates/footer.php';
?>
```

Bagian 4: Create — Menambah Produk Baru

Proses penambahan produk baru melibatkan dua buah file utama [16]:

1. **`produk/create.php`**: Halaman yang berisi form input HTML yang akan diisi oleh pengguna [16].
2. **`produk/proses/store.php`**: File backend murni (tanpa HTML) yang bertugas menerima data kiriman, memvalidasinya, menyimpannya ke database, dan melakukan pengalihan halaman (*redirect*) [16].

Halaman Form: `produk/create.php`

Form ini mengirimkan data menggunakan metode **POST** ke file **`proses/store.php`** [16]. Perhatikan penggunaan operator penggabungan **`?? ''`** untuk mempertahankan nilai input agar tidak hilang saat validasi gagal [18]!

```
<?php
// produk/create.php
require __DIR__ . '/../config/koneksi.php';
require __DIR__ . '/../templates/header.php';

// Memulai session untuk membaca flash error message
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
?>

<link rel="stylesheet" href="/style.css">

<div class="container mt-4" style="max-width: 600px;">
    <h2 class="mb-4 text-center">Tambah Produk Baru</h2>

    <!-- Menampilkan Error dari Session (Flash Message) -->
    <?php if (isset($_SESSION['error'])): ?>
        <div class="alert alert-danger">
            <?= $_SESSION['error']; ?>
        </div>
        <?php unset($_SESSION['error']); // Hapus segera setelah ditampilkan ?>
    <?php endif; ?>

    <!-- Form Action mengarah ke proses/store.php -->
    <form action="./proses/store.php" method="post" enctype="multipart/form-data" class="card p-4 shadow-sm">

        <div class="mb-3">
            <label for="nama" class="form-label">Nama Produk</label>
            <!-- Form retention: jika validasi gagal, nama yang diketik sebelumnya tidak hilang -->
            <input type="text" name="nama" id="nama" class="form-control" value="<?= htmlspecialchars($_POST['nama'
```

```
        </div>

        <div class="mb-3">
            <label for="harga" class="form-label">Harga Produk</label>
            <input type="number" name="harga" id="harga" class="form-control" value="<?= htmlspecialchars($_POST['h
        </div>

        <div class="mb-3">
            <label for="stock" class="form-label">Stock Awal</label>
            <input type="number" name="stock" id="stock" class="form-control" value="<?= htmlspecialchars($_POST['s
        </div>

        <div class="mb-3">
            <label for="kategori" class="form-label">Kategori</label>
            <select name="kategori" id="kategori" class="form-select">
                <option value="">-- Pilih Kategori --</option>
                <option value="Makanan">Makanan</option>
                <option value="Minuman">Minuman</option>
                <option value="Elektronik">Elektronik</option>
            </select>
        </div>

        <div class="mb-4">
            <label for="image" class="form-label">Upload Gambar Produk</label>
            <input type="file" name="image" id="image" class="form-control">
        </div>

        <div class="d-flex justify-content-between align-items-center">
            <a href="/produk/index.php" class="btn btn-secondary">Kembali</a>
            <button type="submit" class="btn btn-primary px-4">Submit</button>
        </div>
    </form>
</div>

<?php require __DIR__ . '/../templates/footer.php'; ?>
```

Pemroses Data Sisi Server: `produk/proses/store.php`

Ini adalah logika inti backend Anda. Ketika tombol submit ditekan, data mendarat di sini.

> ⚠️ **Aturan Emas Alur Pemrosesan (Pola PRG)**
> File pemroses **TIDAK BOLEH menghasilkan output HTML sama sekali** [15]. Setelah query simpan berhasil dijalankan, file pemroses harus langsung melakukan pengalihan (*redirect*) menggunakan fungsi `header("Location: ...")` [16, 17].
>
> Mengapa? Jika tidak di-redirect, lalu pengguna secara tidak sengaja menekan tombol **F5 (Refresh)** di browser, maka form kiriman POST akan terkirim ulang dan produk tersebut akan tersimpan ganda di database [17]. Alur ini disebut **PRG (Post-Redirect-Get)** [17]. Jangan lupa letakkan perintah `exit;` setelah `header()` agar PHP berhenti mengeksekusi sisa kode di bawahnya [17]!

Berikut adalah alur bersih yang menggabungkan validasi input dasar Pertemuan 11 dengan struktur modern yang Anda miliki:

```
<?php
// produk/proses/store.php
require __DIR__ . '/../../config/koneksi.php';

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// 1. AMBIL DATA DARI FORM POST
$nama     = $_POST['nama'] ?? '';
$harga    = $_POST['harga'] ?? 0;
$stock    = $_POST['stock'] ?? 0;
$kategori = $_POST['kategori'] ?? '';

// 2. VALIDASI SISI SERVER (SERVER-SIDE VALIDATION)
// Mengapa divalidasi lagi padahal di HTML sudah dipasang 'required'?
// Karena validasi HTML di browser sangat mudah diakali atau dimatikan lewat Inspect Element DevTools [19].
if (empty($nama) || $harga <= 0 || $stock < 0) {
    $_SESSION['error'] = "Harap isi semua kolom dengan benar. Harga dan stock tidak boleh negatif!";
    header("Location: ../create.php"); // Kembalikan ke halaman form
    exit; // Wajib dihentikan [17]
}

// 3. PROSES SIMPAN KE DATABASE (MENGGUNAKAN PREPARED STATEMENT)
// Di repositori Anda, Anda sudah menuliskan cara yang SANGAT AMAN (Menghindari SQL Injection) [20, 21].
$sql  = "INSERT INTO products (nama, harga, stock, kategori) VALUES (?, ?, ?, ?)";
$stmt = $conn->prepare($sql);

if ($stmt) {
    // Ikat parameter (s = string, i = integer)
    $stmt->bind_param("siis", $nama, $harga, $stock, $kategori);

    if ($stmt->execute()) {
        $_SESSION['sukses'] = "Produk berhasil ditambahkan!";
        header("Location: ../index.php"); // Sukses, alihkan kembali ke daftar produk
        exit;
    } else {
        $_SESSION['error'] = "Gagal menyimpan ke database: " . $stmt->error;
        header("Location: ../create.php");
        exit;
    }
} else {
    $_SESSION['error'] = "Kesalahan Query Persiapan: " . $conn->error;
    header("Location: ../create.php");
    exit;
}
?>
```

Bagian 5: Mengapa Kode Repositori Anda Jauh Lebih Keren?

Di dalam file pemroses repositori Anda, Anda menulis baris-baris validasi gambar dan pengikatan query database yang belum ada di modul pelajaran dasar. Mari kita beri apresiasi dan jelaskan secara singkat kelebihannya:

1. Pertahanan Terhadap SQL Injection [20]

Modul standar menuliskan query SQL secara mentah seperti:

```
// SANGAT BAHAYA! Rentan dibobol hacker lewat SQL Injection [20, 21]
$sql = "INSERT INTO products (nama) VALUES ('$nama')";
```

Sedangkan di repositori, Anda menggunakan **`prepare()`** dan **`execute()`** [21]:

```
// SANGAT AMAN! MySQL memisahkan struktur query dari isi data [21]
$sql = "INSERT INTO products (nama, harga) VALUES (?, ?)";
$stmt = $conn->prepare($sql);
```

Dengan cara ini, meskipun hacker mencoba menginputkan karakter aneh seperti **`' OR '1'='1`** di form input, MySQL akan tetap menganggapnya sebagai teks biasa dan bukan sebagai instruksi query [20, 21].

2. Penanganan Upload Gambar yang Aman

Anda menerapkan pengecekan tipe MIME menggunakan **`getimagesize()`** dan pengecekan whitelist extension (**`jpeg`**, **`webp`**, **`png`**). Ini adalah langkah pengamanan standar tinggi untuk memastikan pengguna tidak mengunggah file script berbahaya (seperti **`.php`** backdoor) yang bisa merusak server Anda!

Ringkasan Intisari Pertemuan 11

Topik	Poin Utama yang Harus Diingat
Fungsi `conn`	Merupakan objek jembatan aktif ke MySQL. Wajib dipanggil untuk setiap operasi database [9].
Object-Oriented Style	Menggunakan penulisan kelas **`conn->query()`** yang jauh lebih bersih dan sesuai standar industri modern dibandingkan gaya prosedural **`mysqli_query(conn, ...)`** [8].
`fetch_all()`	Mengubah hasil query database menjadi array asosiatif utuh sehingga bisa langsung direnderer menggunakan perulangan **`foreach`** murni layaknya array biasa [12].
Pola PRG	Alur wajib: **Post (Form) → Redirect (Header) → Get (Tampil Halaman)** [17]. Berfungsi mencegah duplikasi data produk akibat penekanan tombol refresh (F5) [17].
`exit;`	Harus selalu ditulis langsung di bawah baris **`header("Location: ...")`** agar PHP tidak kecolongan mengeksekusi script di bawahnya sebelum browser berpindah halaman [17].

Topik	Poin Utama yang Harus Diingat
Validasi Lapis Server	Validasi di backend PHP adalah filter keamanan wajib karena validasi sisi klien (HTML required) mudah dilewati oleh pengguna biasa [19].

Modul Pertemuan 11 kustom ini sekarang sepenuhnya sinkron dengan kode modern yang sedang Anda bangun di repositori **`supermarket-php`**. Anda dapat menggunakannya sebagai acuan utama agar tidak lagi merasa kebingungan saat menyelaraskan antara instruksi modul bootcamp dan praktik coding nyata Anda! Selamat belajar!